

HyMeKo: One Hypergraph Substrate for Representing and Learning from Structured Systems

From declarative hypergraph structure to structural-prior learning.

PhD Seminar · Kato Laboratory · Csaba Hajdu · June 2026 · *a primer-style handout: it starts from basic graph theory*

1. Introduction — why graphs, and why that is not enough

Much of the world is naturally described by **relations between things**: people who trust or distrust each other, joints connected in a robot, atoms bonded in a molecule, layers wired in a neural network. The standard tool for relations is a *graph*: dots (*vertices*) joined by lines (*edges*). Graphs are everywhere because they are simple and well understood.

But an ordinary edge joins *exactly two* vertices, and many real relations are ***n*-ary** — they bind several things at once. “Alice, Bob and Carol co-authored a paper” is one three-way fact, not three separate pairwise friendships. If we force it into a graph we draw three edges (Figure 1, left) and silently lose the information that it was *one* joint event. A **hypergraph** keeps it as a single *hyperedge* that contains all three (Figure 1, right).

This talk follows one idea through to its consequences. We take the hypergraph seriously as the basic object and ask two questions about it: (i) *how do we describe* a structured system so that one description can drive many tools (simulators, robot formats, modelling languages, tensor libraries)? and (ii) *how do we learn* from that same structure? HyMeKo answers both with a single substrate — *the structure the framework compiles is the structure the model reasons over*.

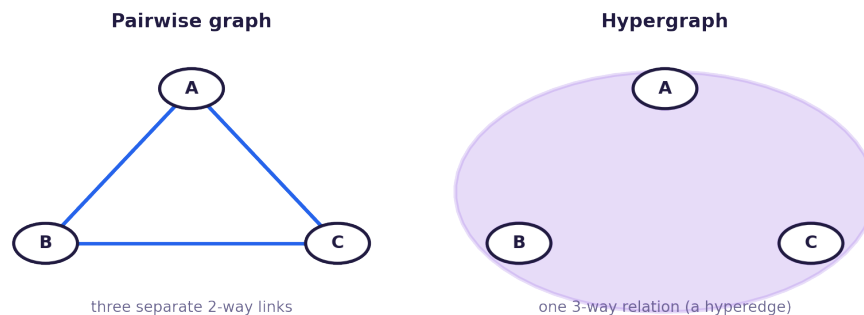


Figure 1: A pairwise graph records three 2-way links; a hypergraph records one 3-way relation. Pairwise encodings lose the identity of group relations.

2. A short tour of graph theory (from the basics)

Vertices, edges, neighbours, degree

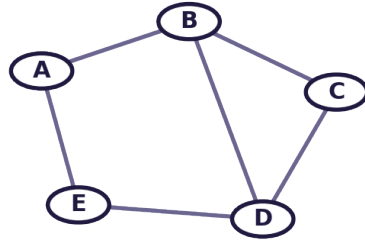
A **graph** is a pair $G = (V, E)$: a set of **vertices** (nodes) V and a set of **edges** E , where each edge $\{u, v\}$ joins two vertices. If edges have a direction we write $(u \rightarrow v)$ and call the graph *directed*. The **neighbours** of a vertex v are the vertices joined to it, $N(v)$, and its **degree** is $\deg(v) = |N(v)|$ — how many connections it has. In Figure 2, $\deg(B) = 3$ (neighbours A, C, D).

Walks, paths and cycles

A **walk** is a sequence of edges followed end-to-end, e.g. $A-B-C$; its *length* is the number of edges. A **path** is a walk that never repeats a vertex. A **cycle** is a closed path that returns to where it started, e.g. $A-B-D-E-A$. These three notions — walk, path, cycle — are central later: HyMeKo’s models read structure off the *cycles* of a graph.

Two matrices that store a graph

With $n = |V|$ vertices and $m = |E|$ edges, a graph is stored either as an **adjacency matrix** $A \in \{0, 1\}^{n \times n}$ (with $A_{ij} = 1$ when i, j are joined) or as an **incidence matrix** $B \in \{0, 1\}^{n \times m}$ (with $B_{ie} = 1$ when vertex i belongs to edge e). Edges may also carry a **weight** (a number) or, as we use next, a **sign**.



vertices $\{A..E\}$, edges $\{AB, BC, CD, DE, EA, BD\}$

Figure 2: A small undirected graph. Example path $A-B-C$; example cycle $A-B-D-E-A$; $\deg(B) = 3$.

A few more kinds of graph

Several special shapes recur. A graph is **complete** (K_n) when every pair of vertices is joined — it has $\binom{n}{2}$ edges, the densest case. It is **bipartite** when the vertices split into two sides with edges only *across* (never within) a side; the signed “star” form of a hyperedge (§6) is exactly a bipartite graph. A **tree** is a connected graph with no cycles; on n vertices it has exactly $n - 1$ edges, the sparsest connected case. A **subgraph** keeps a subset of the vertices and edges. Edges may also be **directed** (an arc $u \rightarrow v$, as in dataflow or a kinematic chain) or **weighted** (a number per edge); the **sign** we use next is the simplest weight, just ± 1 .

Connectivity, distance, and the Laplacian

Two vertices are **connected** if some walk joins them, and a maximal connected piece is a **component**. The **distance** between two vertices is the length of a shortest path between them, and the largest such distance is the graph’s **diameter**. Collecting the degrees on a diagonal gives the **degree matrix** D , and $L = D - A$ is the **graph Laplacian** — the discrete analogue of a second derivative, whose spectrum encodes how connected and how “smooth” the graph is. The Laplacian, and its higher-order Hodge relatives, returns later: it is how the vision variant measures curvature and flow over a hypergraph.

3. Signed graphs & structural balance

A **signed graph** adds a sign to every edge, $\sigma : E \rightarrow \{+, -\}$ — think trust/distrust, friend/foe, agreement/disagreement. Structural balance is *not* part of a standard first graph-theory course, so we spell it out.

The triangle cases. Take three people and the relationships among them. Up to relabelling there are four sign patterns (Figure 3): all-positive (+++), two-positive-one-negative (++−), one-positive-two-negative (+−−), and all-negative (−−−). A triangle is **balanced** when the *product* of its three edge signs is +. This matches social intuition (Heider, 1946): “a friend of my friend is a friend” (+++) and “an enemy of my enemy is a friend” (+−−) are the comfortable, balanced cases, whereas “a friend of my friend is my enemy” (−−−) and “three mutual enemies” (−−−) feel tense and are unbalanced.

From triangles to any cycle. The same rule extends to every cycle: $\sigma(C) = \prod_{e \in C} \sigma(e)$, and C is balanced iff this product is +.

Harary’s balance theorem (1953). A signed graph is *fully* balanced exactly when its vertices can be split into two camps so that every + edge stays *within* a camp and every − edge runs *between* them — “two factions: friends inside, enemies across.”

Weak balance, and how balanced real graphs are. Heider’s original notion (1946) was psychological; Cartwright and Harary (1956) gave it the graph-theoretic form above. Davis (1967) then relaxed it: a graph is **weakly balanced** (*clusterable*) when no cycle has *exactly one* negative edge — which permits all-negative triangles (three mutual rivals in three different camps) and lets the vertices fall into $k \geq 2$ camps, not only two. No real social or trust network is perfectly balanced; the standard measure of how far off it is, is the **frustration (line) index** — the fewest edge-sign flips needed to reach balance. Empirically these networks sit close to balance, and that closeness is exactly what makes the missing-sign question answerable at all.

Why this drives prediction. Real networks are only *approximately* balanced, but the pull toward balance is strong, so the sign of a *missing* edge is usually the one that keeps the cycles through it balanced. **Signed link-sign prediction** hides some edge signs and asks a model to recover them; HSiKAN and Gömb learn a soft, data-driven version of exactly this cycle-balance signal — and doing it *honestly*, without peeking at the

answer through the structure, is a recurring theme of the talk.

Signed triangles — balanced when the product of the three signs is +

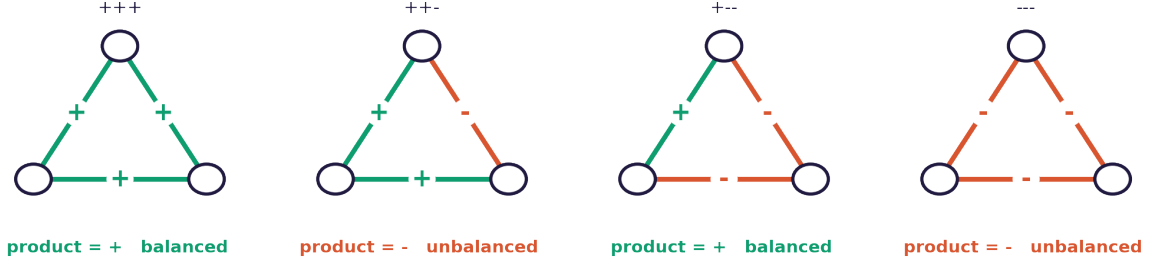


Figure 3: The four signed triangles. Balanced $\Leftrightarrow$ the product of the three edge signs is + (the $+++$ and $+- -$ cases); the other two are unbalanced.



Figure 4: A k -cycle (closed: returns to its start) and a k -walk (open path); green +, red -. A cycle's sign is the product of its edge signs.

4. From graphs to hypergraphs

A **hypergraph** $H = (V, E)$ relaxes the “two endpoints” rule: each **hyperedge** $e \in E$ is simply a subset of vertices, $e \subseteq V$, of any size $|e| = k$ (its *arity*). $k = 2$ recovers an ordinary graph; $\bar{d}$ denotes the mean arity. Two standard ways turn a hypergraph back into something a matrix library can chew on:

- **Star (Levi/Berge) expansion.** Add one “hub” node per hyperedge and connect it to that hyperedge's members; the signed vertex $\times$ hyperedge incidence is $B \in \{-1, 0, +1\}^{|V| \times |E|}$. Cost is $O(|E| \bar{d})$ — linear in the arity.
- **Clique expansion.** Replace each hyperedge by a clique (join all its members pairwise). This recovers a plain graph but costs $O(|E| \bar{d}^2)$ — the quadratic “blow-up” (a k -way relation becomes $\binom{k}{2}$ edges).

HyMeKo uses the star expansion precisely to keep cost linear and to preserve each relation's identity (Figure 5).

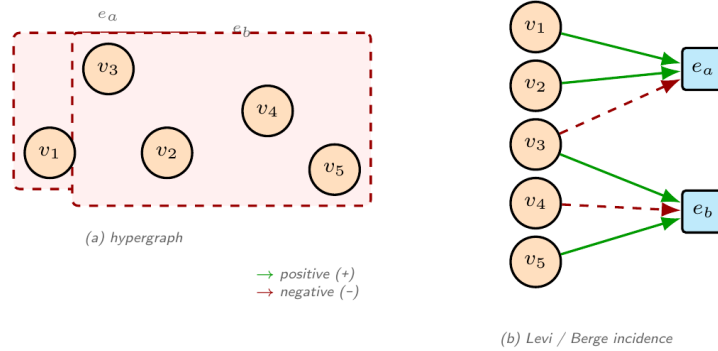


Figure 5: A signed hypergraph (hyperedges e_a, e_b over $v_1 \dots v_5$) and its Levi/Berge incidence — the star expansion (green +, red -).

5. Kolmogorov–Arnold representation & KANs

A classical theorem (Kolmogorov–Arnold, 1957) says any continuous function of many variables can be written using only *single-variable* functions and addition:

$$f(x_1, \dots, x_n) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=1}^n \psi_{q,p}(x_p) \right).$$

A **Kolmogorov–Arnold Network (KAN)** turns this into a learnable model: instead of fixed activations (like ReLU) on the *nodes*, it places *learnable single-variable functions on the edges*. We represent each such function as a **spline** — a smooth curve pinned to a few control points (here a Catmull–Rom spline, Figure 6). HSiKAN puts these learnable splines on the *signed cycle and walk tuples* of the hypergraph, so the graph’s own structure becomes the model’s inductive prior.

Why is the theorem surprising? It says the only genuinely multivariate operation you ever need is *addition*: all the “shape” lives in one-dimensional functions $\psi_{q,p}$ and Φ_q . A KAN takes that literally. A conventional **MLP** fixes the activation (ReLU, tanh) and *learns the linear weights* on its edges; a KAN inverts this — the edges carry the *learnable functions*, and there is no separate weight matrix. A function’s parameters are just the heights of a handful of **control points**, and a **Catmull–Rom** spline interpolates smoothly *through* those points with local control (moving one point only perturbs its neighbourhood), so the curve stays interpretable and cheap to evaluate.

This is a good fit for our setting. A signed cycle’s contribution is a fairly smooth function of its tuple of features, and we would rather the model *learn which cycle lengths matter* than assume it in advance. Placing one learnable spline per signed cycle/walk tuple makes the graph’s own structure the inductive prior — the model’s capacity is spent on the shape of the structure-to-sign map, not on a generic dense network.

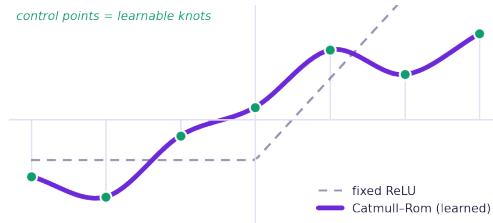


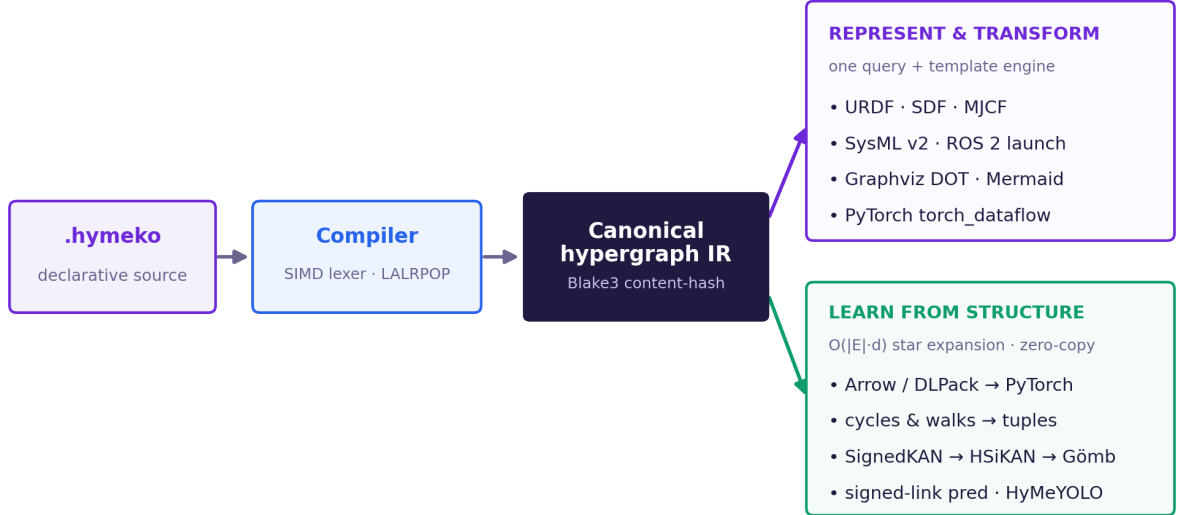
Figure 6: A learnable Catmull–Rom spline activation (vs. a fixed ReLU) — the KAN primitive.

6. One substrate, two payoffs

A declarative `.hymeko` source compiles through a SIMD lexer and an LALRPOP parser to one canonical hypergraph IR with a Blake3 content-hash identity. From that single IR the framework both emits many target formats and feeds the structural-prior learning stack.

How it works — the pipeline

1. **Author** — a declarative `.hymeko` source; geometry, colours and vocabulary reused by reference, not duplicated.
2. **Compile** — a SIMD lexer + LALRPOP parser produce one canonical hypergraph IR with a Blake3 content-hash identity.
3. **Represent** — one query-and-template engine emits URDF, SDF, MJCF, SysML v2, ROS 2 launch, DOT, Mermaid and a PyTorch module, cross-view consistent.
4. **Stream** — the $O(|E| \bar{d})$ star expansion goes zero-copy to PyTorch (Iceoryx2 / Arrow / DLPack); a topology-hash gate ($< 100 \mu\text{s}$) separates structure from weights.
5. **Enumerate** — signed k -cycles and k -walks over the training edges become the structural tuples $(c_3 \cdot c_4 \cdot c_5 \cdot w_2 \cdot w_3)$.
6. **Learn** — SignedKAN $\rightarrow$ HSiKAN $\rightarrow$ Gömb (Clifford-FIR $\rightarrow$ HSiKAN-CR $\rightarrow$ CPML) with Catmull–Rom splines, a per-edge CR highway (`edge_cr`) and quaternion sign-attention.
7. **Audit** — a leakage-audited strict protocol: the cycle pool never sees a test-edge sign; label-shuffle collapses to chance (0.540); 5-seed paired- Δ .



One source · one IR · two payoffs — the graph the framework compiles is the graph the model reasons over.

Figure 7: The HyMeKo workflow: one source $\rightarrow$ one canonical IR $\rightarrow$ two payoffs (represent & transform; learn from structure).

7. Part I — The framework

- Declarative DSL with reuse by reference: a geometry (and colour) defined once, referenced by both visual and collision; the kinematics vocabulary imported once.
- Canonical hypergraph IR with a Blake3 content-hash identity — isomorphic descriptions share one fingerprint.
- One query-and-template engine emits URDF / SDF / MJCF / SysML v2 / ROS 2 / DOT / Mermaid / PyTorch — no recompile per target.
- Zero-copy bridge to PyTorch via Iceoryx2 / Arrow / DLPack, with a topology-hash gate ($< 100 \mu\text{s}$).

Table 1: Star vs. clique materialisation on the same 50-node / 50-edge / 0.30-density hypergraph — the linear vs. quadratic cost of §4 in numbers.

Materialisation	Non-zeros	Extraction
Star $O(E \bar{d})$	1,498	79.7 ms
Clique $O(E \bar{d}^2)$	10,991	496 ms

Table 2: Source size (characters), same robot. HyMeKo reuses geometry/colours by reference; URDF duplicates visual + collision. [†] The full HyMeKo source also encodes joint-control interfaces, sensors and the Gazebo/ros2_control deployment in one file — which plain URDF/MJCF cannot, so the kinematics-only MJCF figure is not a like-for-like comparison.

Kinematic structure	HyMeKo	URDF	MJCF
2-joint connection	71	164	91
SCARA	101	252	129
6-DoF arm (kinematics only)	2,779	5,088	1,733
6-DoF arm (full: + control, sensors) [†]	6,134	multi-file	mech. only

8. Part II — Learning from structure

- Enumerated signed k -cycles and k -walks become inductive features (tuples) — structure as a prior.
- SignedKAN $\rightarrow$ HSiKAN $\rightarrow$ Gömb replaces fixed activations with learnable Kolmogorov–Arnold (Catmull–Rom) splines; HSiKAN adds an arity mixer, a per-edge Catmull–Rom highway (`edge_cr`) and quaternion sign-attention.
- Gömb is a strict three-shell cascade — Clifford-FIR (quaternion/multivector signed filter) $\rightarrow$ HSiKAN-CR $\rightarrow$ CPML — with the cycle pool enumerated over training edges only.

- A vision variant adds Clifford-FIR / quaternion machinery with Forman–Ricci curvature and Hodge decomposition ($\partial^2=0$, 1-homology, discrete Ricci flow).

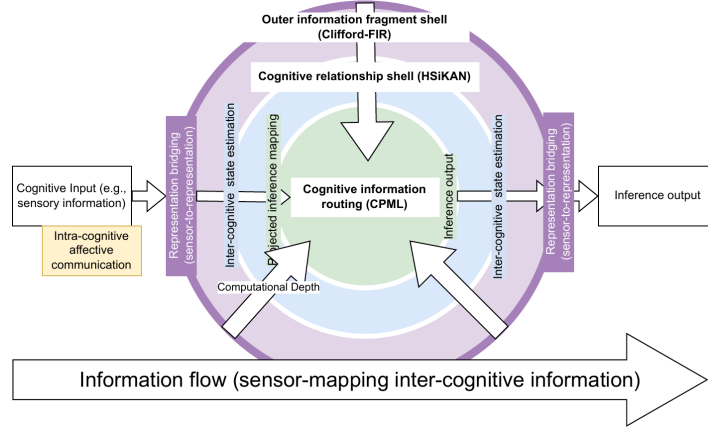


Figure 8: The Gömb structural-prior cascade (Clifford-FIR $\rightarrow$ HSiKAN-CR $\rightarrow$ CPML).

9. Key results (honest framing)

- Competitive-to-leading at $\approx \frac{1}{4}$ of the parameters, with the genuine lead on Epinions — not a flat SOTA claim. (Under the transductive convention, tuned configs reach 0.996/0.993 on Bitcoin.)
- Efficiency: $\sim 30k$ parameters; the win is accuracy-per-parameter. Within-family forward gap (h4 vs. h16, same device) $\sim 3.5\times$ on CPU, $\approx 1\times$ on CUDA.
- Transfer: HyMeYOLO reaches $mAP_{50} \approx 0.90$ (a tie with a ReLU backbone); kinematics recovered from topology alone to ≈ 5 cm.
- Methodology: label-shuffle diagnostic (Gömb $\rightarrow 0.540$ = chance); 5-seed paired- Δ ; every number maps to an on-disk artifact.

Table 3: Signed-graph link prediction, held-out test ROC-AUC (5-seed, Gömb-strict). Baselines are published numbers.

Dataset	Gömb-strict (ours)	SGCN	SiGAT
Bitcoin- α	0.897	0.929	0.903
Bitcoin-OTC	0.915	0.942	0.932
Slashdot	0.902	0.919	—
Epinions	0.943 (FT 0.953)	—	≈ 0.95

10. New result — Cayley-rotor embeddings (2026-06-16)

A flat-parameter, inductive, leakage-free embedding: each item embedding is a learned rotation (a Clifford rotor via the Cayley map) of one shared reference on a sphere — no per-node lookup table. It matches a growing-parameter baseline (DADSGNN) at $\sim 9\text{--}16\times$ fewer parameters, and label-shuffle collapses it to chance (~ 0.50).

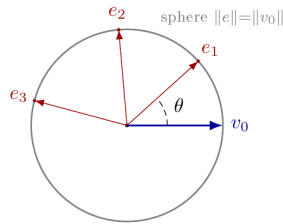


Figure 9: The rotor embedding: each item is a rotation of one reference on a sphere; the angle is the similarity.

11. Contributions & contact

- A canonical hypergraph infrastructure — a DSL plus a content-hashed IR; one source, many targets.

Table 4: Signed-link test ROC-AUC, 5 seeds. Rotor is parameter-flat; the baseline grows. Source: *reports/rotor_multiseed_20260616.jsonl*.

Dataset	Rotor AUROC	Rotor params	DADSGNN AUROC	DADSGNN params
Bitcoin- α	0.867 $\pm$ 0.011	15.8k	0.860 $\pm$ 0.011	142k
Bitcoin-OTC	0.894 $\pm$ 0.006	15.8k	0.880 $\pm$ 0.007	209k
wiki_elec	0.885 $\pm$ 0.002	15.8k	0.896 $\pm$ 0.007	249k

- A structural-prior learning family — cycles and walks as inductive features (SignedKAN / HSiKAN / Gömb), strictly leakage-audited. The same IR is both the engineering transform and the learning substrate.

Csaba Hajdu · Kato Laboratory. Code & docs: https://github.com/kyberszittya/hymeko_framework_rust

12. Slide-by-slide walkthrough

This section mirrors the talk one slide at a time, so the handout can be read on its own after the seminar. Each entry says what the slide shows and why it is there; the conceptual slides (hypergraphs, balance, k -enumeration, leakage) are given a little more room, while the section dividers are kept to a single line.

Slide 1 — Title

The opening slide carries the talk title, the venue (Kato Laboratory), the speaker and the date. It also plants the single thread the whole talk follows: *one substrate, two payoffs* — the idea that the same hypergraph object can both describe a system and be learned from.

Slide 2 — Contributions

Two coupled contributions are stated before any detail, so the audience knows what to hold onto. (i) A canonical hypergraph *infrastructure* — a DSL and a content-hashed IR. (ii) A structural-prior *learning* family that reads a graph’s cycles. The point is that both rest on the *same* IR, and the slide gives the four-block arc of the talk: problem → framework → learning → evidence.

Slide 3 — HyMeKo, in brief

A one-paragraph orientation for newcomers: HyMeKo is a Rust framework plus a small declarative language that compiles a hypergraph description into a canonical intermediate representation, and from that single representation both *transforms* the structure into many target formats and *learns* from it.

Slide 4 — Why hypergraphs?

The motivation. Many real relations bind more than two things at once (n -ary), and squeezing them into ordinary (pairwise) graphs both erases each relation’s identity and inflates the representation. A K_5 illustration makes the cost concrete: turning one k -way relation into pairwise edges needs $\binom{k}{2}$ of them — the $O(d^2)$ clique blow-up. The take-away the audience should leave with is simple: a k -ary relation is one object, and pairwise encoding both multiplies the edge count and forgets that those edges ever belonged together.

Slide 5 — Typical hypergraphs

A gallery of canonical examples — the Fano plane, a generic hypergraph, the complete 3-uniform $K_4^{(3)}$, and a sunflower — each with its hyperedges labelled and its `.hymeko` source shown alongside. The aim is to make “hypergraph” concrete and to show that these are real, compilable fixtures, not just pictures.

Slide 6 — A hyperedge and its signed incidence

Zooms in on a single hyperedge and shows its signed Levi/Berge incidence — the bipartite “star” form with one hub node — using green for + and red for −. This is the representation everything downstream is built on.

Slide 7 — One substrate, two payoffs

The thesis slide. A single canonical IR serves both *representation* (emitting many formats) and *learning* (a structural prior), so that the graph the framework compiles is literally the graph the model reasons over — not two separate artifacts that can drift apart.

Slide 8 — What HyMeKo describes

Surveys the domains the substrate covers in practice: robot mechanisms and kinematics, model-based systems engineering / SysML, signed social graphs, and neural-network dataflow — evidence that the abstraction is general, not bespoke to one field.

Slide 9 — Goals at two levels

Separates the goals into the *language* level (an ergonomic, reusable DSL) and the *infrastructure* level (a fast compiler, a canonical IR, and a zero-copy runtime), so the contributions can be judged on each.

Slide 10 — The HyMeKo workflow — at a glance

A single-page map of the whole system: a `.hymeko` source compiles through the lexer/parser to the canonical IR, which then fans out to the two payoffs — represent & transform on one side, learn from structure on the other. It is the mental model for everything that follows.

Slide 11 — Part I divider

A section divider that opens the framework half of the talk.

Slide 12 — A declarative language for hypergraphs

Introduces the DSL: hyperedges are first-class, nodes nest into hierarchical paths, arcs carry $+/ -$ signs and role tags. A *compilable* `robot.hymeko` example shows the key ergonomic idea — a geometry (and colour) is defined once and *referenced* by both the visual and the collision, rather than duplicated.

Slide 13 — From source text to a hypergraph IR

Walks through the compiler: a SIMD-accelerated lexer and an LALRPOP (LR(1)) parser produce a canonical IR whose nodes carry a Blake3 content-hash identity (so isomorphic descriptions share one fingerprint), with symbol interning and the star/clique expansions available on demand.

Slide 14 — Star vs clique: the efficiency argument

Quantifies §4’s linear-vs-quadratic point on one fixed 50-node / 50-edge / 0.30-density hypergraph: the star expansion needs 1,498 non-zeros and 79.7 ms, the clique expansion 10,991 and 496 ms. The two tensor heatmaps make the density difference visible.

Slide 15 — A zero-copy bridge to the GPU

Explains how the star expansion streams into PyTorch with no copying — via Iceoryx2 shared memory, Apache Arrow and DLPack — and how a topology-hash gate ($< 100\mu\text{s}$) tells structural updates apart from weight streams, so only what changed is re-sent.

Slide 16 — Query-driven transforms: one graph, many targets

Shows the query-and-template engine emitting URDF, SDF, MJCF, SysML and a PyTorch module from the one IR. The $N \times M$ pairwise-converter problem collapses to a hub-and-spoke, and every emitted view is guaranteed consistent with the others.

Slide 17 — How compact is the DSL?

A character-count comparison (HyMeKo vs URDF vs MJCF) on a 6-DoF arm, with the honest framing developed live: HyMeKo wins by reusing geometry/colours by reference where URDF duplicates visual + collision, and the full *deployable* system is one `.hymeko` file versus a multi-file ROS/Gazebo bundle. A screenshot shows the generated robot simulated in MuJoCo.

Slide 18 — Built like a system, not a script

The engineering discipline behind the code: a SysMLv2 model, a `CORE.YAML` protection contract that marks the read-only core, and a plan-first workflow — evidence that the framework is maintained as a system.

Slide 19 — Part II divider

A section divider that opens the science half of the talk.

Slide 20 — From structure to inductive prior

The hinge from Part I to Part II: a graph’s cycles and walks are turned into tuples that act as structural features, and the cycle-arity *compass* is shown as a fingerprint of a mechanism family. Notably the model itself is also expressed in `.hymeko`.

Slide 21 — The model family: SignedKAN, HSiKAN, Gömb

Lays out the progression of models and names the deployable configuration — Clifford-FIR $\rightarrow$ HSiKAN-CR $\rightarrow$ CPML — so the later detailed slides have a map to sit in.

Slide 22 — Gömb: a strict structural-prior cascade

Details the three shells (Clifford-FIR $\rightarrow$ HSiKAN-CR $\rightarrow$ CPML) and the one design choice that makes it honest: the cycle pool is enumerated over *training* edges only, so a test edge’s sign can never leak in — a label-shuffle collapses it to chance.

Slide 23 — What is k-enumeration? Why HSiKAN needs it

Explains the structural features themselves: enumerating signed k -cycles and k -walks of each arity, with graphical examples of a closed cycle and an open walk. It also explains why this is the bottleneck and how it is bounded — per-vertex top- k scoring and per-arity caps. In short, k -enumeration is how raw topology becomes a fixed-width feature that the spline layer can consume, and the caps are what keep it tractable on dense graphs.

Slide 24 — Inside HSiKAN

Opens the middle shell: the Catmull–Rom spline activation, the mixed-arity layers fused by an arity mixer, quaternion sign-attention, and the per-edge Catmull–Rom highway (`edge_cr`) that is the strong reference configuration.

Slide 25 — HSiKAN, learned: alpha-k regime and ROC

Shows real inference output rather than a schematic: the learned arity-weight vector α_k (which cycle lengths the model relies on) and the ROC curve with its AUROC.

Slide 26 — Clifford-FIR and quaternion machinery

The outer shell: parallel Clifford-FIR filter banks built on the `hymeko_clifford` autograd backend (multivectors, blade products, a (p, q) metric), with Forman curvature, Hodge decomposition and Bochner — framed through a $V1 \rightarrow V4 \rightarrow IT$ analogy.

Slide 27 — Signed-graph link prediction results

The headline results: held-out 5-seed ROC-AUC across datasets under the Gömb-strict protocol, next to published baselines, presented as a table and a chart with the honest framing (competitive-to-leading at a fraction of the cost, a genuine lead on Epinions, not a flat SOTA claim).

Slide 28 — SOTA range — at a fraction of the cost

The efficiency story: $\sim 30k$ parameters and a win measured in accuracy-per-parameter, the within-family width gap (h4 vs h16), and a note on the Komondor supercomputer used for the large multi-seed runs.

Slide 29 — New result: Cayley-rotor embeddings

The newest result: a flat-parameter, inductive, leakage-free embedding where each item is a learned rotation of one shared reference on a sphere (a Clifford rotor via the Cayley map). It matches the growing-parameter DADSGNN baseline at $9\text{--}16\times$ fewer parameters, and label-shuffle collapses it to chance.

Slide 30 — What is leakage? Why it matters

A teaching slide on the failure mode the protocol guards against: transductive σ -leakage, where a model scores well by secretly reading test-edge signs through the structure rather than predicting them. Concretely: if the cycle pool is built over all edges, a cycle through a held-out edge can already “see” its sign, and a high score then measures memorisation, not generalisation — which is why the next slides are about ruling this out.

Slide 31 — Honesty as a protocol

States the evaluation stance: a strict protocol in which every reported number maps to an on-disk artifact, and claims are framed conservatively.

Slide 32 — How we know it isn’t leakage

The evidence for the previous slide: re-training with the training-edge signs shuffled drives Gömb to 0.540 (chance), which proves its skill comes from genuine structure, not from a memorised table. The logic is a control experiment: destroy the structure (shuffle the signs) and the skill must vanish if it was structural — it does, falling to the chance line, so the signal the model uses is the balance pattern, not a leaked label.

Slide 33 — HyMeYOLO: detections on Cluttered MNIST

The vision transfer: a hypergraph-convolution detector finding digits in Cluttered MNIST, reaching $\text{mAP}_{50} \approx 0.90$ — an honest statistical tie with a ReLU backbone, presented as an *approach* to hypergraph convolution, on a Ricci-curvature / Hodge backbone.

Slide 34 — The bridge

Ties the two halves together: `.hymeko` structure $\rightarrow$ IR $\rightarrow$ sparse incidence tensor $\rightarrow$ structural-prior model, i.e. one DSL describes the robot and the network that reasons about it. The embedded MuJoCo video shows graph-only kinematics tracking the simulator.

Slide 35 — Live demo: UR5e grasping context

The live, on-robot-stack demo (MDPI Technologies Tier-1): one launch command brings up Gazebo + UR5e + MoveIt 2 + RViz 2 plus the HyMeKo grasping-context node, which loads `hymeko_robot.hymeko`, evaluates 6 signed hyperedges at 10 Hz and republishes contextual outputs as ROS topics. The slide pairs the architecture schematic with the recorded MoveIt demo.

Slide 36 — Publications & submissions (2026)

The publication portfolio shown as a one-substrate-many-outputs fan, situating the infrastructure paper and the HSiKAN work-in-progress within the wider body of work.

Slide 37 — Future work

The near-term programme: finishing the σ -masked strict protocol, a runnable `.hymeko $\rightarrow$ torch.nn` round-trip, broader transform targets, larger signed-graph and vision corpora, the HSMM $\rightarrow$ FPGA path, and an in-browser authoring editor.

Slide 38 — Minimum demo & collaboration

The smallest end-to-end demo that proves the thesis, and a concrete collaboration ask: robot-structure learning, encoding morphology and task in HyMeKo and testing whether structural priors help control.

Slide 39 — Acknowledgements

Thanks to Kato Laboratory for hosting the seminar and to the collaborators and co-authors.

Slide 40 — Conclusion & outlook

Recaps the single idea — one substrate, two payoffs — and the bridge between describing a system and learning from it, then looks ahead.